

Evidence: Autonomous 4WD vehicle report.

Mauricio Ruiz Montemayor
IMT
Tecnológico de Monterrey
Monterrey, México
a01284642@tec.mx

Arturo Gutiérrez Irías
IMT
Tecnológico de Monterrey
Monterrey, México
a01721811@tec.mx

Jorge Treviño Gómez
IMT
Tecnológico de Monterrey
Monterrey, México
a01722087@tec.mx

Eduardo de la Rosa Flores
IMT
Tecnológico de Monterrey
Monterrey, México
a01039969@tec.mx

Summary— In this report, the process of the development of an autonomous 4 wheeled vehicle is detailed. The report includes theory used in the design and implementation of mechanics, programming and control systems. Finally, conclusions about the project are given by each team member.

Key words—*Mechatronics, programming, control, I2C, STM32, UART, PID, sensors, steering.*

I. INTRODUCTION

Automation is the future. It offers many benefits and advantages such as reducing risks in dangerous tasks, minimizing human error, especially in processes that require precision, and it paves the way for new high-level job opportunities. The global shift towards automation is undeniable, businesses and entire industries are seeking new tools and ways to automate their systems and people to develop and use these advanced technologies. This is precisely where mechatronics engineers play an essential role.

Mechatronics centers itself between multiple areas of study: mechanics, electronics, control, and programming. There is no area more important than the other, but rather the integration and synergy of all these disciplines. This is the foundation of a mechatronic system and the specialization of a mechatronics engineer.

The focus of the project tasked for the Industrial Automation class is to apply the concepts learned in class in order to develop an autonomous vehicle able to drive in a 10x10m square. This project is also a collaboration with John Deere, who specified that the entire system must be controlled by an STM32F103C8T6 microcontroller from the STM32 family (throughout the report, this microcontroller will be abbreviated to just “STM32”). The choice of this microcontroller, known for its compact size, cost-effectiveness, and processing speed, was influenced by John Deere's consistent use of STM family microcontrollers in their vehicles.

Additionally, MATLAB was used in order to develop control systems and run plant identification. Other software was used for specific tasks such as Proteus for schematic design and

fritzing to demonstrate the electronic wiring in a more visual way.

The primary objective of this report is to showcase how the knowledge acquired throughout the classes was implemented into the project as well as explaining the key parts of our project, and its development process.

This project also has as an objective to follow what we have learned during the social responsibility module, with the intent of making a project that seeks a common good with the society, making it accessible to everyone and making sure that the repercussions it has are mostly positive. Projects like this have the opportunity to make the world a safer and a better place one step at a time.

II. THEORETICAL FRAMEWORK

Steering

In an autonomous vehicle, steering has a key role in multiple areas of the vehicle functionality, for example, in an urban autonomous vehicle, having proper and precise steering can prevent accidents, avoid hazards, and make the vehicle safer overall. In the case of an autonomous utility vehicle used on a farm, steering quickly and correctly can make the entire operation more efficient, leading to reduced costs and faster production time.

There are multiple types of steering, each one having its strengths and weaknesses, and each one of them excels in certain circumstances. One prevalent example is Ackermann steering, which finds widespread use in urban vehicles such as cars and trucks. According to an article published by the *University of Science and Technology Beijing*, Ackermann steering is a type of steering that consists of a pair of driving wheels and a pair of steering wheels. In this type of steering, the translation and rotation of the vehicle are independent of each other, and it tries to avoid turning by slipping the tires. [1] This type of steering tends to be more energy efficient in comparison to other types since it reduces tire scrubbing and employs smoother turns, thereby reducing tire wear.

Differential steering, as explained by Mart3n Kuslits in collaboration with the Brandenburg Institute of Technology, operates on the difference between longitudinal forces on the two sides of a vehicle. This method utilizes traction and braking forces in a controlled manner to steer. [2] There many implementations of differential steering each having a different use, scope, and complexity.

Skid steering represents another steering method commonly observed in utility vehicles like loaders and military tanks. Skid steering is a type of differential steering mostly used in vehicles that have tracks. It allows a vehicle to have a zero-radius turn and is ideal for vehicles that need to have very good maneuverability and operate in tight spaces. It consists of having fixed tracks or wheels and to turn they need tread slippage which is not energy efficient. [3] It is also difficult to control and predict its behavior because of unwanted slippage. [4]

Control

To make the process autonomous, control is needed in order to make it effective. A well-made controller allows a system to have the least amount of error and obtain the best response. Open-loop control is a type of control that does not receive any feedback. This type of control is useful when the output of a system does not change often or when the process is too erratic. Closed loop control uses feedback to minimize and correct the error obtained from the output. The feedback is usually obtained from sensors and this type of control is used when attempting to output an exact reference value. The difference between the two types of control can be seen in Figure 1 and Figure 2 respectively.

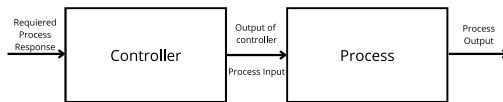


Figure 1 Block diagram of an open loop control system.

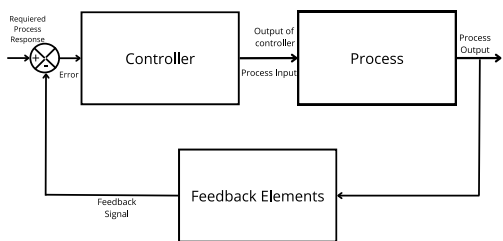


Figure 2 Block diagram of a closed loop control system.

In a closed-loop control system, two distinct types of control loops can be implemented: servocontrol and regulatory control. Servocontrol responds to a change in setpoint. This setpoint or reference can be changed multiple times, and this change usually follows a function in time. On the other hand, regulatory control responds to a change in input or a disturbance, aiming to achieve a reference. Each type of control loop is applied in different situations, but according to the University of Edinburgh, regulatory control is used more than servo control

in the industry. [5] The behaviors of control loops are shown in the following images, Figure 3 for the servo control system, and Figure 4 for the regulatory control system.

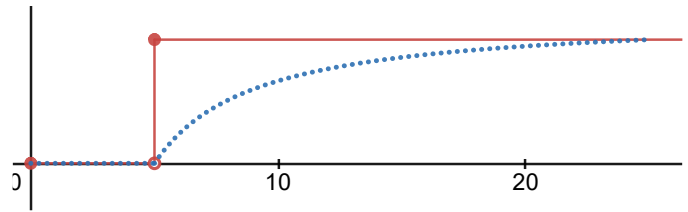


Figure 3 Example of behavior of a servo control system

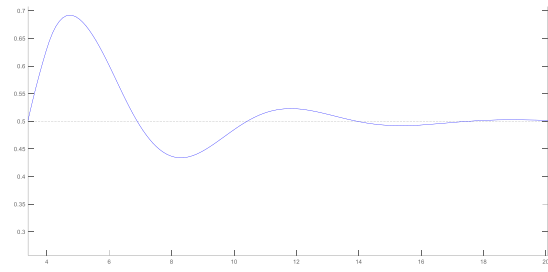


Figure 4 Example of behavior of a regulatory control system.

Most systems are controlled in the Laplace domain, or the frequency domain, and all transfer functions are the products of applying the Laplace transformation to the equation of the behavior of a system. Ideally, when trying to apply control, it is important that the system behaves as a continuous system, this means that the system is analyzed and changes continuously over time. Unfortunately, attempting to make a system continuous will result difficult and expensive in most cases. Therefore, the most common way to work with a system is by making it a discrete system. In this type of system, the changes and analysis are done discretely. When making a system discrete, it is important that the sampling rate of the system is as small as possible so that it emulates a continuous system, allowing the person that is designing the control strategy to implement continuous control strategies. The smaller the sampling rate the better.

Once you obtain your system, the next step is to control it. For this, it is important to choose the right controller or compensator. One of the most popular types of controllers is a PID controller. PID stands for Proportional-Integral-Derivative controller, and each of the parts of the PID has a specific use. Furthermore, not every single part of the controller has to be used, for example a controller that just has an integral and a proportional part is not a PID but a PI.

The proportional part of a PID controller gives variation to the input and is instantaneous, but in most cases, it is not sufficient to control a system. Varying the proportional part of a PID incorrectly can very easily make the controller worse, it can even make a stable system unstable. [6]

The integral action of a PID controller aims to reduce or cancel the difference between the value that is being obtained in the system and the required steady-state value. If the integral action acts too quickly, the controller will require more time to

reach the steady state and there is a higher probability of having a bigger overshoot or oscillation. [6]

The derivative part of a PID is proportional to the slope of the input and has an anticipation effect, its function is to stabilize the loop. It usually reduces the amount of time it takes to reach the steady state which is commonly associated to the number of oscillations on a system. [6]

Programming

Developing a program correctly requires more than just ensuring the code works. A well written code must follow variable nomenclature and employ comments in order to explain what each section of the code does so a third party is able to understand. Optimization of the code is especially important to take into account when wanting to improve efficiency and resource utilization of the microcontroller.

Interrupts are a way to exit the current function running to run a specific section of the code. These interruptions commonly work on external input or are linked to a timer. External interrupts are simple: when an input like a click of a button triggers, it exits the code, runs the code written in the interrupt function, and then the main program is resumed. A timer-based interrupt works similarly to an external interrupt, it uses clock pulses to count the time elapsed and when the count of pulses reaches a value, the interrupt function runs. Interrupts also work to eliminate the bounce of physical elements in a system, such as buttons, as well as filters. External interrupts and timer-based interrupts can also be programmed to work together.

The timer of the microcontroller also allows the output of values that simulate an analog signal. A microcontroller can only output a digital signal, this signal acts as on and off, with its' working voltage and ground. When an analog voltage value is needed, PWM is implemented. PWM stands for pulse-width modulation, it turns on the output for a certain amount of time then turns it off for another. When this is done rapidly, the pulsing voltage emulates a lower voltage, the value of the emulated voltage is directly proportional to the duty cycle. The duty cycle is the ratio of on time to off and is interpreted as a percentage. [7] Figure 5 shows how PWM works and how duty cycle can be visualized.

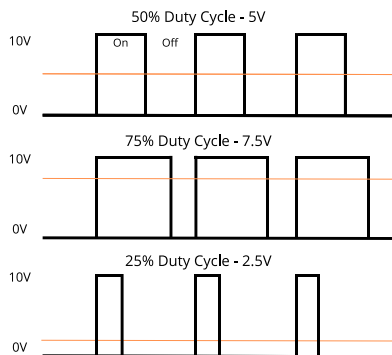


Figure 5 Duty cycles of different PWM signals and their average value (orange line).

There must be special consideration not only for the digital aspects of the system, but also the physical ones as well. When working with analog values from physical components the use of an Analog-to-Digital Converter (ADC) is necessary. Paul Kirvan, technical writer, editor and educator, describes the ADC as the following: “[Talking about ADC] is an electronic process in which a continuously variable, or analog, signal is changed into a multilevel digital signal without altering its essential content. An analog-to-digital converter changes an analog signal that's continuous in terms of both time and amplitude to a digital signal that's discrete in terms of both time and amplitude.” [8]

When wanting to transmit data between two devices or components, communication protocols are needed. One of the most used and simplest communications protocols in the industry is UART, which stands for universal asynchronous receiver/transmitter. UART was one of the first communication protocols, but with the development of new technologies, other communication protocols have gained popularity and now UART is used when simplicity and a low cost is required for low-speed applications. [9] UART uses serial ports to transmit data, and since the protocol is asynchronous, both devices that are attempting to communicate must have very similar clock speeds in order to be compatible. This is a problem other communication protocols do not exhibit. To fix these problems, USART, which stands for Universal Synchronous Asynchronous Receiver Transmitter, was created. The synchronous part of USART allows clock communication between the devices. [10]

The integration of multiple peripherals and controllers often demands additional measures to ensure successful and reliable communication. For these cases, Inter-Integrated Circuits, also called by their initials (I2C), fulfil this requirement. I2C is a communication protocol that allows control of multiple slaves with a single master and have multiple masters controlling a singular slave or multiple slaves. I2C only needs two ports to communicate data, the SDA which stands for serial data and the SCL or SCLK which stands for serial clock. The serial data port is for the master and slave to transmit information and the SCLK carries the clock signal. I2C fixes many of the problems that UART and USART had but does it at a greater monetary cost. [10]

III. MATERIALS

Qty.	Component (Model)
1	STM32F103C8T6 (Bluepill)
1	ST-LINK v2 programmer
2	H-bridges (L2N8N)
4	Encoder disk
4	Encoder sensor (H206)
1	Inertial Measuring Unit (IMU MPU6050)
1	Bluetooth Module (HC-05)

2	Transparent acrylic bases for 4WD chassis with protective coating
8	Acrylic support for gearmotor
4	Metallic spacer with M3 thread
4	Gearmotor 48:1
4	Tire with semi-hard rubber 65 mm x 27mm
6	M3 x 6mm fine thread screw
8	M3 x 30mm fine thread screw
14	M3 fine thread nut
3	SD18650 Li-Po rechargeable battery
1	Battery holder
1	Battery charger
	Dupont Cables

Table 1 Build of materials (BOM) of the project.

IV. METHODOLOGY

Brainstorm and Management

To start the project, the team brainstormed strategies to obtain a solution to the challenge. The solutions discussed were centered around the requirements set by the teachers and the collaborator. Special considerations for simplicity and cost were taken into account. It was decided that the project tasks would be divided into different sections and each section would be tasked to a different team member.

In the end, it was decided that the steering system of the vehicle would be skid-steer, since it allows us to obtain a zero-radius turn and for its simplicity of its mechanical components. We discarded Ackermann steering and conventional differential steering for the same reasons mentioned. It was also decided that the use of 4 DC motors, each attached to a wheel of the vehicle would enable better control of the vehicle. For sensors, an optical encoder speed sensor was chosen to determine the velocity of each wheel, an inertial measurement unit (IMU) to work as a gyroscope that would measure change in angular speed and a magnetometer to determine the direction of the car.

The following Gantt Chart (Figure 6) was created to establish estimated times for each task of the project and to see the critical route. The list of tasks can be seen in Table 2.

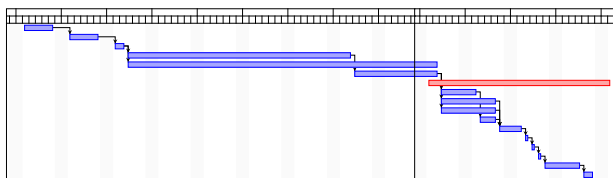


Figure 6 Gantt Chart of the project.

	Name	Duration	Start	Finish	Predecessors
1	Planning	5 days	8/28/23 8:00 AM	9/1/23 5:00 PM	
2	Buy and receive material	5 days	9/4/23 8:00 AM	9/8/23 5:00 PM	1
3	Build car	2 days	9/11/23 8:00 AM	9/12/23 5:00 PM	2
4	Test components	25 days	9/13/23 8:00 AM	10/17/23 5:00 PM	3
5	Write open loop code	35 days	9/13/23 8:00 AM	10/31/23 5:00 PM	3
6	Test open loop control	10 days	10/18/23 8:00 AM	10/31/23 5:00 PM	4
7	Write report	20 days	10/30/23 9:00 AM	11/27/23 9:00 AM	
8	Design and print new enco...	4 days	11/1/23 8:00 AM	11/6/23 5:00 PM	6
9	Read IMU	7 days	11/1/23 8:00 AM	11/6/23 5:00 PM	6
10	Read magnetometer	7 days	11/1/23 8:00 AM	11/6/23 5:00 PM	6
11	Read encoder sensor	3 days	11/7/23 8:00 AM	11/6/23 5:00 PM	8
12	Send telemetry	2 days	11/10/23 8:00 AM	11/13/23 5:00 PM	9;10;11
13	Obtain step and random r...	1 day	11/14/23 8:00 AM	11/14/23 5:00 PM	12
14	Find if	1 day	11/15/23 8:00 AM	11/15/23 5:00 PM	13
15	Discretize if	1 day	11/16/23 8:00 AM	11/16/23 5:00 PM	14
16	Make PID	4 days	11/17/23 8:00 AM	11/22/23 5:00 PM	15
17	Rewire System	2 days	11/23/23 8:00 AM	11/24/23 5:00 PM	16

Table 2 Table of tasks used for the creation of the Gantt Chart

Choosing and buying material.

One of the most crucial factors when choosing materials and components was price and availability. The budget is limited due to the fact that this project is elaborated by college students. The time was also limited due to the project having a set due date. This limited the components choices and quality, due to the fact that better components are typically more expensive and require longer time to receive them. In some cases, components were chosen because they were already available from previous projects. The components that were chosen were recommended by teachers, fellow students, and the internet.

The components the team already had in hand were an IMU 6DOF, a Magnetometer, a Bluetooth module, 18650-batteries, an 18650-battery charger, and a 3S battery holder. This is the reason the team did not choose a 9DOF IMU. The other components such as the optical encoders, h bridges and motor were all bought from the internet due to recommendations or previous experience with the components. The complete list of materials can be found on Table 1.

There are two main types of Inertial Measuring Units, in our case our IMU is of 6 degrees of freedom. The IMU works as a gyroscope and as an accelerometer. The gyroscope is used to determine the direction change in direction of the car and to make a correct turn, and the accelerometer is not going to be used for any process. The magnetometer was going to be used to obtain the absolute orientation with the earth's magnetic field as a reference. The data of both sensors was planned to be fused to obtain better readings and to eliminate the disadvantages of each sensor. The encoders were used to determine the revolutions per minute of each tire, knowing this value allows us to make a controller that keeps the speed of all tires the same, reducing drift and ensuring the vehicle drives straight.

Building the car

The chassis, wheels and other components of the car came together in a kit that was bought so the integration of these components, screws and nuts were seamless, the only difficulty that was presented was that the kit missed some screws and had to be completed by screws of a different screwdriver head. Assembling peripherals and sensors was a little bit more challenging.

The car kit plates are normally assembled one on top and one on the bottom of the DC motors. This is to give the car more structural stability. To increase the space in the car, the team decided to build one plate on top of the motors and another on top of that plate, to be able to allocate the encoders and H-Bridges more easily. The structural strength of the car was

tested, and it was observed that building it this way did not change the stability of the car much.

The first thing that was installed on the vehicle were both H-bridges, holes for the screws were melted in the acrylic using a soldering iron and nuts were used to fasten the screws in the underside. For the encoder sensors, screw holes also had to be melted into the acrylic. The header pins of the sensors were oriented in a way that didn't allow them to fit in their designated place. The pins had to be desoldered carefully as to not pull the rail away from the board, and then new header were soldered pins to each of the 4 encoder sensors.

Almost at the end of the project, it was decided that the original acrylic base that was bought was going to be changed to a base made from steel with almost the same exact design which can be seen in Figure 7. This decision was taken in order to increase the weight of the vehicle which had been previously tested and showed to improve the behavior of the PID by reducing wheel slipping. The chassis was cut by a plasma cutter from a steel 1/8-inch plate. The residues from the plasma cutter were polished using an angle grinder and the holes for the screws were drilled so they would be bigger and able to pass the screw. It was then covered with insulation tape to prevent any part of the circuit from having a short. Woefully, the insulation tape was not enough, this caused the H-bridge pins to pierce through the tape, touch the steel base and made a short circuit. A cardboard base for the H-Bridge was made to protect the pins from touching the metal plate. The metal plates were then changed back to acrylic due to the vehicle's PID not working correctly. This did not fix the vehicle's PID behavior, but the team decided to keep the acrylic sheets due to time constraints and fear of the metal plate short circuiting.

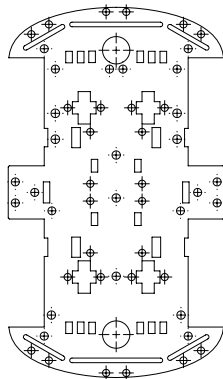


Figure 7 Steel Plate Drawing

Wiring the car and soldering the wires also proved a little difficult because of the lack of space to do so and the length of the wires themselves. The complete system was implemented on a breadboard to ease connecting and disconnecting the wires and components for prototyping, but the number of components and wires made it difficult to identify misconnections and other problems. This ease of connecting and disconnecting unfortunately caused wires that were connected to disconnect with the slightest movement and burned several components including a microcontroller.

Connections and Schematic

To successfully control the car, it is necessary to use 8 GPIO pins as outputs. These pins are used to control the direction of the tires. To control the velocity, it is necessary to use 4 PWM signals. These signals have a resolution of 12 bits which yields a resolution value of 0.0008 V. This value means that we can control increments of 0.8mV. For the sensors, 4 external interrupts were used to read the optical encoders values and one I2C bus. For telemetry an HC-05 Bluetooth module was used using UART. We utilized the internal LED on the STM32 to determine whether both the magnetometer and the IMU were initialized correctly.

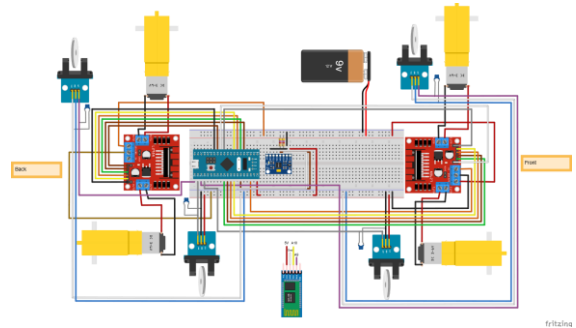


Figure 8 Connection View

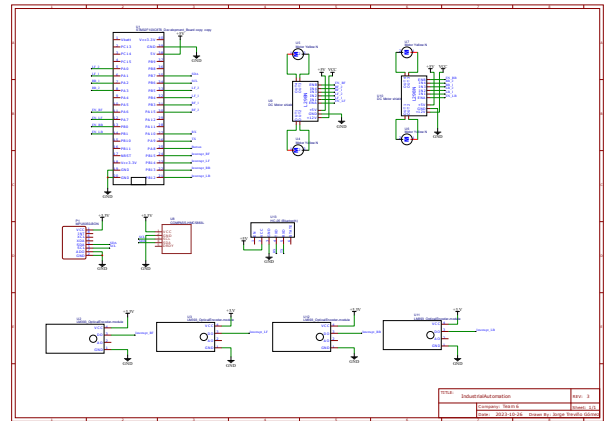


Figure 9 Schematic View

Open loop control

After initializing all the startup functions and variables, the main program begins. It consists of a state machine that has as its states forward and right turn. The first state begins when the user gives the begin instruction via a Bluetooth command. After this command is sent, the state changes from zero to one and it starts measuring time. It was programmed that the vehicle must go forward for a certain amount of time, after the condition is met, the state changes from one to two and begins to turn. This turn is done with feedback from our gyroscope sensor. When it reaches a threshold, it stops and it changes the state from two to one in order to go forward again.

Reading the sensors

1) IMU

To initialize the IMU, the function `IsDeviceReady` is utilized. This function returns `HAL_OK` if the I2C bus is initialized between master and slave. Then, the IMU is initialized. The microcontroller reads the `WHO_AM_I` register which returns the address of the sensor. If this address matches with the one from the datasheet it means its connected. Then the STM32 writes in the `PWR_MGMT_1` register 0 which determines the sensor will be set in normal use. Next, the output data rate and the accelerometer and gyroscope are configured. For the configuration of the sensors, $\pm 2g$ and $\pm 250^\circ/\text{sec}$ are used respectively.

To read the registers of each axis the `HAL_I2C_Mem_Read` function is used. In I2C data consists of a packet of 8 bits. Incoming data consists of a High and Low packet that units to form a single 16-bit value. To do this, the microcontroller reads the High register and shifts it 8 bits and concatenates it with the low register. This value must be multiplied by the precision factor and divided by 2^{15} . This is done for the 3 axes for accelerometer and the 3 axes of the gyroscope.

To turn the vehicle using the gyroscope, the data from the Z axis is integrated, this describes the relative yaw angle. To minimize error, an average of 5 data values is calculated and used. Each value is read every 10 ms and a timer interrupt integrates it every 50ms. To filter the yaw angle, a digital high pass filter was used.

2) Magnetometer

For the magnetometer to work after coding it, calibration is needed. To calibrate the magnetometer, it is rotated along its X, Y and Z axis to obtain readings of the magnetic field on all axes. It is important to do the calibration in the place where the magnetometer is planned to be implemented because magnetic fields vary depending on the surroundings. For example, a calibration done inside a room that was built using steel beams would be different than one done out in the open because the steel beams have a big effect on the magnetic field. The data gathered from this should look like a cloud on a 3D graph which can be seen in Figure 10. The more the raw data looks spherical the better it is. Once the data from revolving the magnetometer around its axes is recollected. It is important to make a sphere from all the data by normalizing it and using averages and moving the data to the center of the graph, this can be seen in **Error! Reference source not found..** Having a fuller sphere, which means collecting more data and more accurately produces better results.

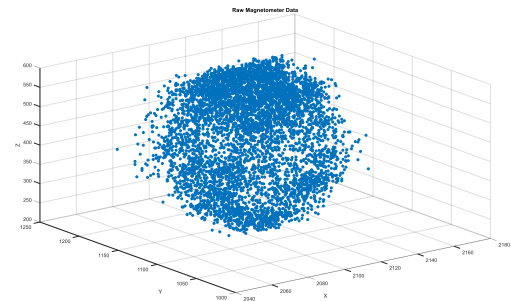


Figure 10 Example of raw data from a magnetometer after rotating it around its axes.

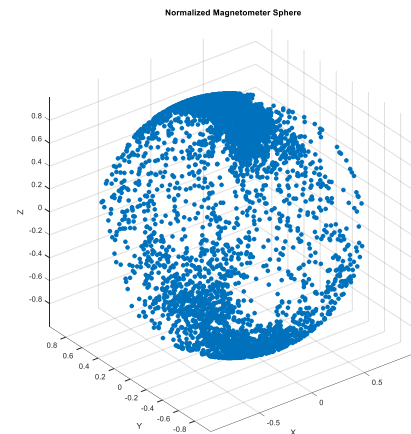


Figure 11 Normalized data from a magnetometer after rotating it around its axes.

Once calibrated, it is also important not only to use the magnetometer where it was calibrated but in the same conditions as it was calibrated and with the least amount of magnetic interference. To do this, in the case of a vehicle, it is recommended to create distance between the magnetometer and other components that are able to affect the magnetic field or create one such as an abundance of cables or motors. This separation can be achieved by creating a platform where the magnetometer could be set up with enough distance from the other components.

The magnetometer was planned to be used in conjunction with the gyroscope to mitigate errors, but it was later realized that the gyroscope had precise all by itself and incorporating the magnetometer would present more trouble and time that was needed for other aspects of the project, so it was decided to not use it but to document the process of calibration, programming and explain why this choice was made.

3) Encoder sensors

Initially, it was decided that the encoder sensors were going to be a module using the FC-03 sensor and an LM393 comparator because of the ease to obtain them, but the sensors had to be changed to the H206 sensor because of a variety of reasons. The main reason was that the LM393 was not correctly configured in the module so it would take noise and convert it into a pulse, which then would indicate greater, incorrect RPMs. This was attempted to be fixed by using capacitors to reduce the noise as well as making a filter in the code using interruptions which did

mitigate the amount of noise in the system but not enough to have a decent reading. It was later discovered that the LM393 had a circuit that was badly designed so it generated these types of problems. For other development boards that have a slower clock speed, such as the Arduino Uno, using these encoders might not prove an issue because the microcontroller is not fast enough to detect the noise. The STM32 lets the user change the clock speed and making it faster or slower, making the clock speed slower would probably fix this issue but it would create many more problems and difficulties, such as having a plant identification and PID that is not similar to a continuous system. The capacitors and the filter were still used with the other encoders.

Telemetry

To transmit data using Bluetooth, the HC-05 Bluetooth module was used. This module supports UART using a baud rate of 9600 bits per second. To enable communication, it was necessary to set the UART clock accordingly. The module was used to receive and transmit data. This allowed the implementation of an emergency stop button in case something went wrong. To receive data, the function `HAL_UART_Receive()` is used, it uses UART handle, a buffer, the message size and a timeout as parameters.

The `HAL_UART_Transmit()` function is used to transmit data. It takes as parameters the UART handle, a buffer, its size and a timeout. It is important to note that the buffer size must be large enough to store all the data.

Plant Identification

To be able to apply control to the system, the behavior of the system needs to be identified. In this case, since the vehicle uses skid steering, the velocity and orientation of the vehicle is going to be controlled by the four motors in each wheel. Having all the wheels in the same speed makes the vehicle go straight, whilst changing the speed of the wheels on one side or the other, makes the car turn. Even though all motors are the same model, and all motors are being supplied with the same H-bridges, the motors behave differently under the same conditions, that is why the identification the 4 motors are needed for the speed controller and the direction controller.

The first identification made was for the speed controller. At the beginning because of what was taught in class, it was chosen to do the identification using a wide-pulse signal. In this type of signal a value in the middle is first sent, then the maximum value and finally the minimum value. This type of signal usually works great when trying to identify a system that is made up of motors. The identification was made using MATLAB's app PIDTuner which helps identify a plant and create PIDs. When identifying a plant using PIDTuner, several parameters need to be added, changed and removed to have the best identification. For the four motors, identifications with over 70% resemblance to the original plant were obtained but unfortunately, the PIDs that were made using the wide-pulse signals and their respective identification didn't work.

The team chose to use another method to identify the system called PRBS, which stands for Pseudo Random binary sequence. This method consists of sending to the plant a random signal of

ons and offs, it usually works great because it simulates a lot of possible behaviors. When making the identification for our system, the PRBS didn't work for all of the wheels and the PIDs did not work completely so this type of identification was discarded.

Finally, the identifications that were chosen were the ones made by using a step signal or a step response. A step response changes the value from the minimum to the maximum and remains at that maximum until the process is stopped or another signal is made. In the Theoretical Framework, in Figure 3, the red line represents a step response. These identifications were chosen because they had a closer resemblance to the original system and had the best working PIDs.

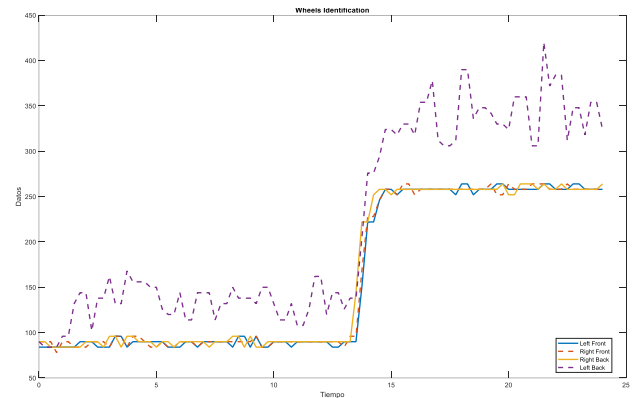


Figure 12 Identification using step response of the four wheels.

Making a PID

With the plant identification done and using the PID tuner app in MATLAB, the system response time and transient behavior were modified to obtain a control system with a response time of approximately 1.5 seconds and a moderate transient behavior. This method was done for the five systems that were identified.

For the motors, a PI was found to be the best option since it minimized overshoot and allowed the system response that was needed to perform control on the vehicle. Similarly, to the motor control, the direction control parameters was also a PI controller.

After obtaining the parameters, a real-life test was performed and by observing the behavior of this test the values of our controller were modified.

V. INDIVIDUAL CONCLUSIONS

1. Jorge Treviño Gómez

This has been the most challenging project I've had in university. It allowed me to explore new areas in coding that I hadn't had the chance to explore before. Using a new microcontroller such as the STM32 and using a lot of its unique functionalities made me realize the benefits of a lower-class programming language. At first, the program of the vehicle seemed simple, just a few motors with a sensor but integrating together and making sure everything works together was tough. This challenge also allowed me to have one of my first

experiences in obtaining the controller parameters for a system. I learned a lot of methods to make this process and the pros and cons of each method. I feel inspired by this challenge and want to do more projects involving control and STM.

As mentioned before, the initial code seemed simple but with a project such as this one we encountered a lot of problems. Most of the time dedicated to this project was troubleshooting. Our problem with the encoders delayed us a few weeks but after all the troubleshooting, the program was done, and the vehicle performed as planned.

One of the main takeaways from this project is the importance of planning and talking to the team. The first meeting we had as a team we discussed a lot of problems that could come true, but we were hopeful that they wouldn't become a reality, but of course they did. We could have more meetings to discuss advancements to allow all team members to grasp the idea of what everyone was doing. I am proud of myself and the team for the work and effort that was poured into this challenge.

2. *Eduardo de la Rosa Flores:*

Engaging in this project has been both challenging and rewarding for me. It stands out as one of the most significant experiences in my academic journey. The project allowed me to put my theoretical knowledge into practice and pushed me to learn extensively, especially in MATLAB and C programming. Working with the STM32 microcontroller was a valuable aspect of the project, providing insights into microcontroller functionalities and enhancing my skills in control and automation. Having to use the STM32 microcontroller presented challenges, even though my prior experience with it. However, overcoming these challenges significantly contributed to my overall learning. Navigating through the complexities of the STM32 improved my problem-solving skills and deepened my understanding of microcontroller-based systems. My main focus in the project was the implementation of PID controllers. This aspect was demanding, especially in the initial stages when understanding the optimal method for system identification was a significant challenge. Despite these difficulties, we successfully used MATLAB's PID Tuner app for system identification. However, obtaining a transfer function was just the beginning; fine-tuning the gains provided by MATLAB was crucial for achieving the desired outcome. In essence, this project served as a comprehensive check of my knowledge about PID controllers and their practical implementation. Advocating for PID design within the team, I faced various complexities, particularly in the early stages. Through persistence and collaboration, we overcame challenges, and the project progressed towards its final objective. Reflecting on the entire process, I am confident that the skills and insights gained will prove invaluable in my future endeavors. Navigating a challenging project, combined with the use of advanced tools and technologies, has contributed significantly to my growth as an engineer. Although the car did not achieve perfection, the understanding gained throughout the process was the real accomplishment. Despite facing challenges in fine-tuning system parameters, I emerged from this experience with a comprehensive grasp of PID controllers and their

implementation. It's essential to note that my current limitations, particularly in C programming skills, played a role in the challenges faced.

Acknowledging this, I am motivated to enhance my programming abilities for better alignment of theory with practice in future projects. The realization that continuous improvement is fundamental in any engineering pursuit serves as a driving force for honing my skills, particularly in C programming, to overcome similar hurdles in the future if using an STM32 microcontroller which functions I understand very well.

3. *Mauricio Ruiz Montemayor*

At the beginning of the semester, this project seemed easy, we were relaxed, and we had other preoccupations. Since we had 15 weeks, it seemed that time was not going to be a factor. One of the first times we had a meeting as a team, we discussed possible problems that we could encounter but were confident that they would not happen because they were very improbable. After working and giving all my effort to this project, I can say that Murphy's Law is in fact true, everything that can go wrong will go wrong.

Even though this project made me really struggle and contemplate my ability as a student and as an aspiring engineer, I am proud to say that we were able to finish it and the final product is also something that I can be proud of.

In a project like this you have the opportunity to learn from a lot of different areas, and because we were part of a team, we also had the opportunity to learn from each other. Everyone in the team had different strengths and weaknesses which led to the tasks of the project to be delegated to the person who was best at it. Unfortunately, I am not the best in the team programming, I am not bad, but simply not the best, so I didn't get the opportunity to program as much as I would have liked during this project. Still, I learned about programming by asking my teammates and teachers and truly think I have become a better programmer. It was also fun to learn new things that I had no idea existed and that will definitely make my life easier in the future.

This challenge had lots of highs and lows but the main lessons that I learned are that not everything can be perfect, it is important to trust your team, be resilient, and most importantly have fun. Many times, during this project it was really easy to give up but what kept us going and with our spirits high was laughing at a simple joke and acknowledging that we were doing something that we love, after all, there is a reason why we study this.

Even though things did not turn out as planned, I am really happy they didn't because it worked as a learning experience. I am also glad I was able to be in a team where I had the opportunity to make stronger friendships and meet a new friend. Lastly, I want to thank my teachers, Cristina Aparicio and Salvador Leal for being excellent at their job and for making this project possible.

4. Arturo Gutierrez Irías

This Project definitely lived up to the name “challenge” and was the first big endeavor we have undertaken that can be said to be truly mechatronic. I learned from this project the true challenge of being a mechatronic engineer which is integration. One can learn and master all the disciplines: mechanics, electronics, control, and programming but really the hardest thing is trying to put it all together and working together as a team.

It was a project that seemed extremely simple at first, and I believe that’s what got us. Before researching well and looking too deep into the implications of various systems, we had already somewhat planned out what we wanted to do. We bought everything and divided the work according to the disciplines. We did not think that neither the programming aspect nor control would be so hard, and we had also taken the electronics part for granted (before being told it had to be with a stm32).

Maybe being an engineer is 30% design and 70% troubleshooting, or maybe we’re not good engineers yet. We honestly learned a lot from the troubleshooting, everything that could’ve gone wrong, went wrong, but through fixing it we learned things that we couldn’t even have imagined were relevant to the project.

I think my biggest takeaway from this project is not the technical lessons we took or how we solved each unforeseen problem we got. The most important thing I learned is how essential good planning, adaptability and communication is. We tried to plan the whole project without really knowing the details and that left us in a disparity between workloads when some parts were harder than others but really that could’ve been fixed with better communication on our part. I really wanted to involve myself more in the coding and control aspects of the project but because we didn’t share what we had been working on during the project, it was too late to involve myself without being a hindrance, so I had to work on other parts of the project. Another lesson I take from this is the importance of parallel work. We thought things would take much less time than they took so we planned to do things after other parts of the project had been finished. At the end of the day, we tried to do everything at the last minute and by changing what we already had, everything stopped working, we should have instead worked on it in an adaptable way so that everything would be integrated before the final stretch.

I think all four of us take big lessons from what happened these last 15 (really 10) weeks and have already started to think about how we can improve on what we can avoid what went wrong and improve on what we did right for next semester.

VI. FINAL CONCLUSION

At the culmination of this mechatronics project, we recognize that it was a tough challenge, one we didn’t expect. Figuring out all the details about a completely new microcontroller and IDE, troubleshooting electronics, and

designing multiple PID controllers through trial and error and overall, integrating all the systems.

This project taught us the essential importance of planning things meticulously and communicating in an effective and continuous manner. Engineering is not just its technical aspects, and this challenge taught us that. Each of us had their personal experiences and lessons but we all learned about resilience and enjoying the process, even when times get tough.

Looking back, we acknowledge that we really had no idea about how tough this challenge would be and how different it would be from our initial plan. We also acknowledge that we could’ve done a lot of things better and have learned from it.

This project, with all the good and the bad, is paving the way for our future as mechatronics engineers. It has taught us that the strength of engineering does not lie just in design, but in collaboration, adaptability, and continuous improvement. We come out of this challenge very differently of what we expected, but proud that we were able to pull things through and our commitment to the project.

VII. REFERENCES

- [1] H. Zhang, Y. Zhang, C. Liu and Z. Zhang, "Energy efficient path planning for autonomous ground vehicles with," *Robotics and Autonomus Systems*, vol. 162, no. 104366, p. 7, 2023.
- [2] M. Kuslits, "Analysis and Optimisation of a New Differential Steering Concept," Logos Verlag Berlin, Berlin, 2022.
- [3] J. Martínez, J. Morales, J. García and A. García-Cerezo, "Analysis of Tread ICRs for Wheeled Skid-Steer," *IEEE Access*, p. 547, 2023.
- [4] C. Wen Zhu, E. Hill, M. Biglarbegan, A. Gadsden and J. Cline, "Smart agriculture: Development of a skid-steer autonomous robot," *Robotics and Autonomous Systems*, vol. 162, no. 104364, p. 2, 2023.
- [5] University of Edinburgh, "Controller Design and System Modelling," [Online]. Available: <https://www.homepages.ed.ac.uk/jwp/control06/course/restricted/course/third/course/controller.html>.
- [6] D. Lequesne, *Practical PID Handbook*, EDP Sciences, 2022.
- [7] J. Upadhyay, "What is a PWM signal?," Circuit Bread, 14 February 2022. [Online]. Available: <https://www.circuitbread.com/ee-faq/what-is-a-pwm-signal>.
- [8] P. Kirvan, "What is analog-to-digital conversion (ADC)?," TechTarget, July 2022. [Online]. Available: <https://www.techtarget.com/whatis/definition/analog-to-digital-conversion-ADC>.
- [9] Rohde & Schwarz, "Understanding UART," [Online]. Available: https://www.rohde-schwarz.com/us/products/test-and-measurement/essentials-test-equipment/digital-oscilloscopes/understanding-uart_254524.html.

- [10] Spark Fun, "I2C," 8 July 2013. [Online]. Available: <https://learn.sparkfun.com/tutorials/i2c/all>.
- [11] Rajguru Electronics, "Electropeak," 16 November 2020. [Online]. Available: <https://5.imimg.com/data5/VQ/DC/MY-1833510/lm393-motor-speed-measuring-sensor-module-for-arduino.pdf>.
- [12] QST, "3-Axis Magnetic Sensor," February 2016. [Online]. Available: <https://www.datasheet-pdf.com/PDF/QMC5883L-Datasheet-QST-1309218>.

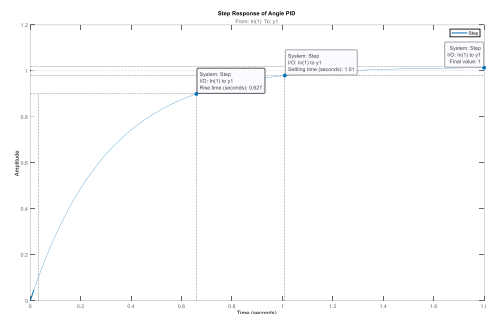


Figure 13 Angle PID

VIII. SUPPORTING MATERIAL



Image 1 Image of the car when the building process started.



Image 3 Image of base manufactured in steel.

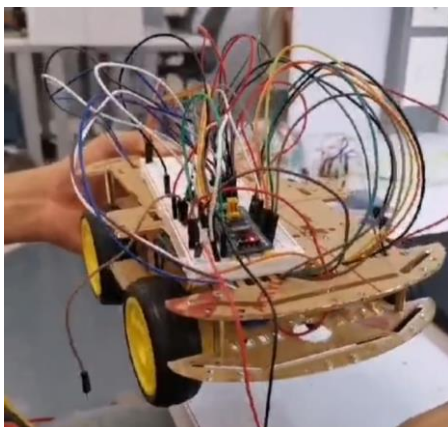


Image 2 Image of the car fully built.

Code

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file           : main.c
 * @brief          : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2023 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file

```

```

    * in the root directory of this software component.
    * If no LICENSE file comes with this software, it is provided AS-IS.
    *
    *****
    */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include "stdio.h"
#include "Directions.h"
#include "PIDpwm.h"
#include "math.h"
/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
I2C_HandleTypeDef hi2c1;

TIM_HandleTypeDef htim1;
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;
TIM_HandleTypeDef htim4;

UART_HandleTypeDef huart1;

/* USER CODE BEGIN PV */

/* IMU Adresses */
uint8_t IMU_ADDRESS = 0b11010000; //0xD0
uint8_t MAG_ADDRESS = 0x1A;

/* Variables de IMU6050 Acelerometro */
uint8_t accel_raw_data[6];

float accel_x;
float accel_y;
float accel_z;

/* Variables de IMU6050 Giroscopio */

```

```

uint8_t gyro_raw_data[6];

float gyro_x;
float gyro_y;
float gyro_z;

float GyroMean[5] = {};
float gyro_mean;

float GyroValuesx = 0;
float GyroValuesy = 0;
float GyroValuesz = 0;

float Gyro_X_calibration = 0;
float Gyro_Y_calibration = 0;
float Gyro_Z_calibration = 0;

/*Angulos*/
float yawangle = 0;
float yawangle_new;
float prev_yaw = 0;
float yaw_reference = -90;
float filteredYaw;
float YawOffsetValue;
uint8_t state = 0;
float pidanlge_to_use;

volatile uint8_t pulses_rf, pulses_lf, pulses_rb, pulses_lb;

float pulsesperturn = 20.0;

float rpm_rf, rpm_lf, rpm_rb, rpm_lb;

float current_tick_rf, current_tick_lf, current_tick_rb, current_tick_lb;

float pid_output_lf, pid_output_rf, pid_output_lb, pid_output_rb;
float Pid_angle_pwm_left, Pid_angle_pwm_right;

float pwm_rf, pwm_lf, pwm_rb, pwm_lb;

float distance_rf, distance_lf, distance_rb, distance_lb, distance_average;

PIDpwm_Controller pidMotor_rf, pidMotor_lf, pidMotor_rb, pidMotor_lb, pidAngle;

float samplingTime = 250.0;

float setpoint = 3.4;

/*Array For Commands*/
char Commands[3];

/*Transmission array*/
char rpmdata[200] = {0};

/* Magnetometer variables*/

```

```

uint8_t array[2]={0x01, 0x1D};
uint8_t array2[6];
int16_t MagX,MagY,MagZ;
float YawMag, angulo2;
float x_center = 2624.1; /* Valor de x_center de MATLAB */;
float y_center = 2561.8; /* Valor de y_center de MATLAB */;
float z_center = 210.8500; /* Valor de z_center de MATLAB */;
float radio_promedio = 153.39167; /* Valor del radio promedio calculado en MATLAB */;
float x_norm, y_norm, z_norm;
float distancia;

// Variables para almacenar los máximos y mínimos de la calibración
char datos[20];

uint8_t count = 0;
uint32_t time_counter = 1;
uint8_t flag;

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_TIM3_Init(void);
static void MX_USART1_UART_Init(void);
static void MX_TIM4_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM1_Init(void);
static void MX_NVIC_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */
void MPU_init(void){
    /*IMU 60050 configuration*/
    uint8_t check, data;

    /*WHO_AM_I register, returns 0x58 if connected*/
    HAL_I2C_Mem_Read(&hi2c1, IMU_ADDRESS, 0x75, 1, &check, 1, 100);

    if(check == 104){
        /*PWR_MGMT_1 register. Write 0 for normal use*/
        data = 0;
        HAL_I2C_Mem_Write(&hi2c1, IMU_ADDRESS, 0x6B, 1, &data, 1, 100);

        /*SMPRT_DIV register*/
        data = 0x07;
        HAL_I2C_Mem_Write(&hi2c1, IMU_ADDRESS, 0x19, 1, &data, 1, 100);

        /*Acceleration Register configuration*/
        HAL_I2C_Mem_Read(&hi2c1, IMU_ADDRESS, 0x1C, 1, &data, 1, 100);

        uint8_t precission_selector = 0;
    }
}

```

```

    /*
    * 0 for +- 2g
    * 1 for +- 4g
    * 2 for +- 8g
    * 3 for +- 16g
    */
    data = (data & ~(0b11<<3));
    data = (data | (precision_selector << 3));

    HAL_I2C_Mem_Write(&hi2c1, IMU_ADDRESS, 0x1C, 1, &data, 1, 100);

    /*Gyro Register configuration*/
    HAL_I2C_Mem_Read(&hi2c1, IMU_ADDRESS, 0x1B, 1, &data, 1, 100);

    uint8_t precision_selector_gyro = 0;
    /*
    * 0 for +- 250 deg/s
    * 1 for +- 500 deg/s
    * 2 for +- 1000 deg/s
    * 3 for +- 2000 deg/s
    */
    data = (data & ~(0b11<<3));
    data = (data | (precision_selector_gyro << 3));

    HAL_I2C_Mem_Write(&hi2c1, IMU_ADDRESS, 0x1B, 1, &data, 1, 100);

}

}

void MPU_Read_accel(void) {

    /*
    * 0x3B ACCEL_XOUT_H
    * 0x3C ACCEL_XOUT_L
    * 0x3D ACCEL_YOUT_H
    * 0x3E ACCEL_YOUT_L
    * 0x3F ACCEL_ZOUT_H
    * 0x40 ACCEL_ZOUT_L
    * */

    /*Read registers in order*/
    HAL_I2C_Mem_Read(&hi2c1, IMU_ADDRESS, 0x3B, 1, accel_raw_data, 6, 1000);

    /*Precision value in g's
    * change if value in MPU_init changes
    * 2
    * 4
    * 8
    * 16
    * */
    float g_precision = 2;

    accel_x = (float) (((int16_t) (accel_raw_data[0] << 8 | accel_raw_data[1])) *
g_precision/32768);
    accel_y = (float) (((int16_t) (accel_raw_data[2] << 8 | accel_raw_data[3])) *
g_precision/32768);

```

```

    accel_z = (float) (((int16_t) (accel_raw_data[4] << 8 | accel_raw_data[5])) *
g_precision/32768);

    /*Conversion from g's to m/s^2*/
    accel_x = accel_x * 9.81;
    accel_y = accel_y * 9.81;
    accel_z = accel_z * 9.81;
}

void MPU_Read_gyro(void) {

    /*
    * 0x43 ACCEL_XOUT_H
    * 0x44 ACCEL_XOUT_L
    * 0x45 ACCEL_YOUT_H
    * 0x46 ACCEL_YOUT_L
    * 0x47 ACCEL_ZOUT_H
    * 0x48 ACCEL_ZOUT_L
    * */

    if(count < 5){

        /*Read registers in order*/
        HAL_I2C_Mem_Read(&hi2c1, IMU_ADDRESS, 0x43, 1, gyro_raw_data, 6, 1000);

        /*Precision value in degs/s
        * change if value in MPU_init changes
        * 250
        * 500
        * 1000
        * 2000
        * */
        float gyro_Precision = 250;

        gyro_x = (float) (((int16_t) (gyro_raw_data[0] << 8 | gyro_raw_data[1])) *
gyro_Precision/32768);
        gyro_y = (float) (((int16_t) (gyro_raw_data[2] << 8 | gyro_raw_data[3])) *
gyro_Precision/32768);
        gyro_z = (float) (((int16_t) (gyro_raw_data[4] << 8 | gyro_raw_data[5])) *
gyro_Precision/32768);

        /*Gyro values adjusted with calibration values*/
        gyro_x += Gyro_X_calibration;
        gyro_y += Gyro_Y_calibration;
        gyro_z += Gyro_Z_calibration;

        GyroMean[count] = gyro_z;
        count++;

    }

    if(count == 5){
        /*Moving average of 5*/
        gyro_mean =
(GyroMean[0]+GyroMean[1]+GyroMean[2]+GyroMean[3]+GyroMean[4])/5;
        count = 0;
    }
}

```

```

void Gyro_calibration(void) {

    uint16_t counter = 0;
    uint32_t StartTime_cal = HAL_GetTick();
    uint32_t ElapsedTime_cal = 0;

    /*Calibrate value for 9 seconds*/
    while(ElapsedTime_cal < 9000){

        MPU_Read_gyro();
        GyroValuesx += gyro_x;
        GyroValuesy += gyro_y;
        GyroValuesz += gyro_z;

        counter++;

        uint32_t CurrentTime_cal = HAL_GetTick();
        ElapsedTime_cal = CurrentTime_cal - StartTime_cal;
    }

    /*Calibration values, average of gyros values*/
    Gyro_X_calibration = -1 * GyroValuesx/counter;
    Gyro_Y_calibration = -1 * GyroValuesy/counter;
    Gyro_Z_calibration = -1 * GyroValuesz/counter;
}

void MPU_GetYaw(uint8_t elapsed){
    uint8_t alfa = 0.98;

    yawangle_new = prev_yaw + gyro_z * elapsed/1000;

    yawangle = (1-alfa) * prev_yaw + (alfa) * yawangle_new;

    prev_yaw = yawangle_new;

    if(yawangle <= -360){
        yawangle = 0;
    }
    if(yawangle > 360){
        yawangle = 0;
    }

    //filteredYaw = alfa*(filteredYaw + gyro_z * elapsed/1000) + (1-alfa)*YawMag;
}

void Mag_init(void) {

    HAL_I2C_Mem_Write(&hi2c1, MAG_ADDRESS, 0x08,1, &array[0], 1, 100);
    HAL_I2C_Mem_Write(&hi2c1, MAG_ADDRESS, 0x09,1, &array[1], 1, 100);
}

void Mag_read(void) {
    HAL_I2C_Mem_Read(&hi2c1, MAG_ADDRESS, 0x06,1, array2, 1, 100);
    if(array2[0]&0x01){
        HAL_I2C_Mem_Read(&hi2c1, MAG_ADDRESS, 0x00,1, array2, 6, 100);
        MagX = array2[1] << 8 | array2[0];
    }
}

```

```

        MagY = array2[3] << 8 | array2[2];
        MagY = array2[5] << 8 | array2[4];
    }
    float x_cent = (float)MagX - x_center;
    float y_cent = (float)MagY - y_center;
    float z_cent = (float)MagZ - z_center;

    distancia = sqrt(x_cent * x_cent + y_cent * y_cent + z_cent * z_cent);
    if (distancia != 0) { // Evitar división por cero
        x_norm = (x_cent / distancia); /* radio_promedio;
        y_norm = (y_cent / distancia); /* radio_promedio;
        z_norm = (z_cent / distancia); /* radio_promedio;
    }
    distancia = sqrt(x_norm * x_norm + y_norm * y_norm + z_norm * z_norm);
    YawMag = atan2(y_norm, x_norm) * 180 / M_PI;
}
/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_TIM3_Init();
    MX_USART1_UART_Init();
    MX_TIM4_Init();
    MX_TIM2_Init();
    MX_TIM1_Init();

    /* Initialize interrupts */
    MX_NVIC_Init();
    /* USER CODE BEGIN 2 */

    /*Check if MPU6050 is connected*/
    HAL_StatusTypeDef ret = HAL_I2C_IsDeviceReady(&hi2c1, IMU_ADDRESS, 10, 100);

```

```

/*Check if QMC5883 is connected*/
HAL_StatusTypeDef ready = HAL_I2C_IsDeviceReady(&hi2c1, MAG_ADDRESS, 10, 1000);

if(ret == HAL_OK /*&& ready == HAL_OK*/){
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 0);
    char transmission_buffer[17];
    uint8_t transmission_size = sprintf(transmission_buffer,"Device connected \n");

    HAL_UART_Transmit(&huart1,&transmission_buffer,transmission_size, 500);
}
else{
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 1);
}

Mag_init();
MPU_init();

Gyro_calibration();

HAL_TIM_Base_Start_IT(&htim4);
HAL_TIM_Base_Start_IT(&htim2);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_4);

uint32_t start_time, elapsed_time, current_time;

/*Initialize PID functions with its obtained Kp, Ti, Td and reference values*/
/*
    PIDpwm_Init(&pidMotor_lf,0.6241f,10.0f,0.025f,-270.0f, 270.0f);
    PIDpwm_Init(&pidMotor_rf,2.915f,10.00f,0.025f,-270.0f, 270.0f);
    PIDpwm_Init(&pidMotor_lb,1.369f,0.2538f,1.846f,-270.0f, 270.0f);
    PIDpwm_Init(&pidMotor_rb,0.0f,0.1283f,0.0f,-270.0f,270.0f);
*/
/*
    PIDpwm_Init(&pidMotor_rf, 1.7513f, 285.5875f, 0.14992f, -300.0f, 300.0f);
    PIDpwm_Init(&pidMotor_lf, 1.9756f, 361.081f, 0.15439f, -300.0f, 300.0f);
    PIDpwm_Init(&pidMotor_rb, 2.4901f, 324.3345, 0.16109f, -300.0f, 300.0f);
    PIDpwm_Init(&pidMotor_lb, 2.131f, 380.4173f, 0.14994f, -300.0f, 300.0f);
*/

pidMotor_rf.setpoint = 3.4;
pidMotor_lf.setpoint = setpoint;
pidMotor_rb.setpoint = setpoint;
pidMotor_lb.setpoint = setpoint;

pidAngle.setpoint = 0;

/*
    PIDpwm_Init(&pidMotor_rf, 0.90f, 0.0f, 0.0f, -300.0f, 300.0f);
    PIDpwm_Init(&pidMotor_lf, 1.0f, 0.0f, 0.0f, -300.0f, 300.0f);
    PIDpwm_Init(&pidMotor_rb, 0.8f, 0.0f, 0.0f, -300.0f, 300.0f);
    PIDpwm_Init(&pidMotor_lb, 0.9f, 0.0f, 0.0f, -300.0f, 300.0f);
*/
PIDpwm_Init(&pidMotor_rf, 140.0f, 400.0f, 0.0f, -4.0f, 4.0f,3.4);

```

```

PIDpwm_Init(&pidMotor_lf, 140.0f, 400.0f, 0.0f, -4.0f, 4.0f, 3.4);
PIDpwm_Init(&pidMotor_rb, 140.0f, 400.0f, 0.0f, -4.0f, 4.0f, 3.4);
PIDpwm_Init(&pidMotor_lb, 140.0f, 400.0f, 0.0f, -4.0f, 4.0f, 3.4);
/*Set point definition*/

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    HAL_UART_Receive(&huart1, &Commands, 3, 100);

    /*State Machine for going forward and turning right.*/
    switch(state){

        case 1:
            //Forward path
            //Forward();

            pwm_lf = 4095;
            pwm_lb = 4095;
            pwm_rb = 4095;
            pwm_rf = 4095;

            /*Set PWM value to the PIDs PWM output value*/
/*
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1,
pwm_lf); //pid_output_rf);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2,
pwm_rf); //pid_output_lf);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3,
pwm_lb); //pid_output_lb);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4,
pwm_rb); //pid_output_rb);
*/

            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pid_output_lf);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, pid_output_rf);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, pid_output_lb);
            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, pid_output_rb);

/*
            if(yawangle > yaw_reference_straightLine){
                RightTurn();

                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1,
pid_output_lf*(4095.0/300));
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2,
pid_output_rf*(4095.0/300));
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3,
pid_output_lb*(4095.0/300));
                __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4,
pid_output_rb*(4095.0/300));
            }
*/

```

```

        if(yawangle > yaw_reference_straightLine){
            LeftTurn();
        }
        */
        break;

    case 2:
        //Right turn

        pwm_lb = 3500;
        pwm_rb = 3500;
        pwm_rf = 3500;
        pwm_lf = 3500;

        /*Set PWM value to the PIDs PWM output value*/
        /*
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_lf);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, pwm_rf);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, pwm_lb);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, pwm_rb);
        */

        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pid_output_lf);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, pid_output_rf);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, pid_output_lb);
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, pid_output_rb);

        break;

    default:
        Stop();

        break;
}

current_time = HAL_GetTick();
elapsed_time = current_time - start_time;

/*State machines logic */
if(state == 0){
    flag = 1;
    /*Start Command*/
    if(Commands[0] == 'S'){
        start_time = HAL_GetTick();
        elapsed_time = 0;

        PIDangle_Init(&pidAngle,30.0f,0.0f,-45.0f,45.0f);

```

```

        pidMotor_rf.setpoint = setpoint;
        pidMotor_lf.setpoint = setpoint;
        pidMotor_rb.setpoint = setpoint;
        pidMotor_lb.setpoint = setpoint;

        flag = 0;

        state++;
    }
}
else if(state == 1){
    flag = 0;
    /*Drive Forward for 5 seconds*/ // Change THIS!!
    if(distance_average > 950){

        pidAngle.setpoint = yaw_reference;

        setpoint = 3;

        pidMotor_rf.setpoint = setpoint;
        pidMotor_lf.setpoint = setpoint;
        pidMotor_rb.setpoint = setpoint;
        pidMotor_lb.setpoint = setpoint;

        start_time = HAL_GetTick();
        elapsed_time = 0;
        state++;
    }
}
else if(state == 2){

    /*Turn right until yaw reference value*/
    if(yawangle >= yaw_reference * 1.05 && yawangle <= yaw_reference * 0.95){
        yaw_reference += -90 + (-yawangle + yaw_reference);
        distance_average = 0;
        flag = 1;

        //yaw_reference += -90;
        Stop();
        /*Recalibrate at every stop*/
        YawOffsetValue += yawangle;
        Gyro_calibration();

        setpoint = 3.4;

        pidMotor_rf.setpoint = setpoint;
        pidMotor_lf.setpoint = setpoint;
        pidMotor_rb.setpoint = setpoint;
        pidMotor_lb.setpoint = setpoint;

        pidAngle.setpoint = yawangle;
    }
}

```

```

        start_time = HAL_GetTick();
        elapsed_time = 0;
        state = 1;
    }
}

/*Stop Command*/
if(Commands[0] == 'E'){
    Stop();

    state = 0;

    pwm_lb = 0;
    pwm_rb = 0;
    pwm_rf = 0;
    pwm_lf = 0;

    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, 0);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, 0);

    setpoint = 0;

    pidMotor_rf.setpoint = setpoint;
    pidMotor_lf.setpoint = setpoint;
    pidMotor_rb.setpoint = setpoint;
    pidMotor_lb.setpoint = setpoint;

    pidAngle.setpoint = 0;
}

uint8_t num_chars = sprintf(rpmdata,"%u, %.2f,
%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f\n",state,pidMotor_rf.setpoint,
pid_output_lf, pid_output_rf, pid_output_lb, pid_output_rb, rpm_lf, rpm_rf, rpm_lb,
rpm_rb, yawangle);
//uint8_t num_chars = sprintf(rpmdata,"%u,%.2f,%.2f,%.2f,%.2f\n",rpm_lf, rpm_rf,
rpm_lb, rpm_rb);

//uint8_t num_chars =
sprintf(rpmdata,"%u,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f\n", rpm_lb, rpm_rf,
rpm_lb, rpm_rb, rpm_lf, rpm_rf, rpm_lb, rpm_rb, yawangle);

HAL_UART_Transmit(&huart1, &rpmdata, num_chars, 100);

// uint16_t Datas = sprintf(Dats,"%u,%u\n",Intervalo, rpm_lb);

//HAL_UART_Transmit(&huart1, &Dats, Datas, 10);

}

/* USER CODE END 3 */
}

```

```

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
     */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief NVIC Configuration.
 * @retval None
 */
static void MX_NVIC_Init(void)
{
    /* EXTI15_10_IRQn interrupt configuration */
    HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
    /* TIM4_IRQn interrupt configuration */
    HAL_NVIC_SetPriority(TIM4_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM4_IRQn);
    /* TIM2_IRQn interrupt configuration */
    HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
    /* TIM3_IRQn interrupt configuration */
    HAL_NVIC_SetPriority(TIM3_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM3_IRQn);
}

```

```

/**
 * @brief I2C1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_I2C1_Init(void)
{
    /* USER CODE BEGIN I2C1_Init 0 */

    /* USER CODE END I2C1_Init 0 */

    /* USER CODE BEGIN I2C1_Init 1 */

    /* USER CODE END I2C1_Init 1 */
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN I2C1_Init 2 */

    /* USER CODE END I2C1_Init 2 */

}

/**
 * @brief TIM1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM1_Init(void)
{
    /* USER CODE BEGIN TIM1_Init 0 */

    /* USER CODE END TIM1_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    /* USER CODE BEGIN TIM1_Init 1 */

    /* USER CODE END TIM1_Init 1 */
    htim1.Instance = TIM1;
    htim1.Init.Prescaler = 0;
    htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim1.Init.Period = 65535;
    htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

```

```

htim1.Init.RepetitionCounter = 0;
htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM1_Init 2 */

/* USER CODE END TIM1_Init 2 */

}

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{
    /* USER CODE BEGIN TIM2_Init 0 */

    /* USER CODE END TIM2_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    /* USER CODE BEGIN TIM2_Init 1 */

    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 7200 - 1;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 2500 - 1;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

```

```

    if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM2_Init 2 */

    /* USER CODE END TIM2_Init 2 */

}

/**
 * @brief TIM3 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM3_Init(void)
{
    /* USER CODE BEGIN TIM3_Init 0 */

    /* USER CODE END TIM3_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_OC_InitTypeDef sConfigOC = {0};

    /* USER CODE BEGIN TIM3_Init 1 */

    /* USER CODE END TIM3_Init 1 */
    htim3.Instance = TIM3;
    htim3.Init.Prescaler = 0;
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim3.Init.Period = 4096 - 1;
    htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim3) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim3, &sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
    {
        Error_Handler();
    }
    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 0;
    sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;

```

```

    if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
    {
        Error_Handler();
    }
    if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_4) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM3_Init 2 */

    /* USER CODE END TIM3_Init 2 */
    HAL_TIM_MspPostInit(&htim3);

}

/**
 * @brief TIM4 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM4_Init(void)
{
    /* USER CODE BEGIN TIM4_Init 0 */

    /* USER CODE END TIM4_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};

    /* USER CODE BEGIN TIM4_Init 1 */

    /* USER CODE END TIM4_Init 1 */
    htim4.Instance = TIM4;
    htim4.Init.Prescaler = 7200 - 1;
    htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim4.Init.Period = 100-1;
    htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim4) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim4, &sClockSourceConfig) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

```

```

    if (HAL_TIMEx_MasterConfigSynchronization(&htim4, &sMasterConfig) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM4_Init 2 */

    /* USER CODE END TIM4_Init 2 */

}

/**
 * @brief USART1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART1_UART_Init(void)
{
    /* USER CODE BEGIN USART1_Init 0 */

    /* USER CODE END USART1_Init 0 */

    /* USER CODE BEGIN USART1_Init 1 */

    /* USER CODE END USART1_Init 1 */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 9600;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART1_Init 2 */

    /* USER CODE END USART1_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */

```

```

__HAL_RCC_GPIOC_CLK_ENABLE();
__HAL_RCC_GPIOD_CLK_ENABLE();
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();

/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);

/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                  |GPIO_PIN_15, GPIO_PIN_RESET);

/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5, GPIO_PIN_RESET);

/*Configure GPIO pin : PC13 */
GPIO_InitStruct.Pin = GPIO_PIN_13;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

/*Configure GPIO pins : PA0 PA1 PA2 PA3
                        PA15 */
GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                  |GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/*Configure GPIO pins : PB12 PB13 PB14 PB15 */
GPIO_InitStruct.Pin = GPIO_PIN_12|GPIO_PIN_13|GPIO_PIN_14|GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/*Configure GPIO pin : PA8 */
GPIO_InitStruct.Pin = GPIO_PIN_8;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/*Configure GPIO pins : PB3 PB4 PB5 */
GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){

```

```

if(htim -> Instance == TIM2){

    if(flag == 0){

        rpm_rf = (current_tick_rf * 1000 ) / (pulsesperturn * samplingTime);
        rpm_lf = (current_tick_lf * 1000 ) / (pulsesperturn * samplingTime);
        rpm_rb = (current_tick_rb * 1000 ) / (pulsesperturn * samplingTime);
        rpm_lb = (current_tick_lb * 1000 ) / (pulsesperturn * samplingTime);

        distance_rf = current_tick_rf * 20.7345/pulsesperturn;
        distance_lf = current_tick_lf * 20.7345/pulsesperturn;
        distance_rb = current_tick_rb * 20.7345/pulsesperturn;
        distance_lb = current_tick_lb * 20.7345/pulsesperturn;

        distance_average += (distance_rf + distance_lf + distance_rb +
distance_lb)/4;

        distance_rf = 0;
        distance_lf = 0;
        distance_rb = 0;
        distance_lb = 0;

        current_tick_rf = 0;
        current_tick_lf = 0;
        current_tick_rb = 0;
        current_tick_lb = 0;

        pid_output_lf = PIDpwm_Compute(&pidMotor_lf, rpm_lf, 1);
        pid_output_rf = PIDpwm_Compute(&pidMotor_rf, rpm_rf, 2);
        pid_output_lb = PIDpwm_Compute(&pidMotor_lb, rpm_lb, 3);
        pid_output_rb = PIDpwm_Compute(&pidMotor_rb, rpm_rb, 4);

        // PID M1 //

    }

}

if(htim -> Instance == TIM4){

    if(count == 4){
        /*Integrate every 50ms*/
        MPU_GetYaw(50);
        Pid_angle_pwm_left = -1* PIDangle_Compute(&pidAngle, yawangle);
        Pid_angle_pwm_right = - Pid_angle_pwm_left;
        pid_output_lf += Pid_angle_pwm_left;
        pid_output_lb += Pid_angle_pwm_left;
        pid_output_rf += Pid_angle_pwm_right;
        pid_output_rb += Pid_angle_pwm_right;
        //count = 0;

    }

}

```

```

        /*Readd gyro value every 10ms*/
        MPU_Read_gyro();
    }

}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    /*
    * @pin 15 Right front wheel
    * @pin 14 Left front wheel
    * @pin 13 Right back wheel
    * @pin 12 Left back wheel
    */

    if(GPIO_Pin == GPIO_PIN_15){
        //Right front wheel
        pulses_rf++;
        current_tick_rf++;
    }
    if(GPIO_Pin == GPIO_PIN_14){
        //Left front wheel
        pulses_lf++;
        current_tick_lf++;
    }
    if(GPIO_Pin == GPIO_PIN_13){
        //Right back wheel
        pulses_rb++;
        current_tick_rb++;
    }
    if(GPIO_Pin == GPIO_PIN_12){
        //Left back wheel
        pulses_lb++;
        current_tick_lb++;
    }
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**

```

```

* @brief Reports the name of the source file and the source line number
* where the assert_param error has occurred.
* @param file: pointer to the source file name
* @param line: assert_param error line source number
* @retval None
*/
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

Code 1 Main code of the vehicle.

```

/*
 * PID.c
 *
 * Created on: Nov 12, 2023
 * Author: eduar
 */
#include "PIDpwm.h"
#include "Directions.h"

//definicion de funciones
void PIDpwm_Init(PIDpwm_Controller* pid, float Kp, float Ti, float Td, float
min_output, float max_output, float setpoint){
    pid->Kp = Kp;
    pid->Ki = Kp/Ti;
    pid->Kd = Kp * Td;
    //inicializar variables tipo float
    pid->setpoint = setpoint;
    pid->integral = 0.0f;
    pid->errorAnterior = 0.0f;
    pid->medicionSensor_Anterior = 0.0f;
    //limites del PID
    pid->min_output = min_output;
    pid->max_output = max_output;

    pid->Prev_output = 0.0;

}

float PIDpwm_Compute(PIDpwm_Controller* pid, float medicionSensor, int MotorVariable) {

    float voltageOutput;
    float output2;

    float errorpid = pid->setpoint - medicionSensor;
    pid->integral += errorpid * 250;
    float derivativo = (errorpid - pid->errorAnterior)/250;
    float output = pid->Kp * errorpid + pid->Ki * pid->integral + pid->Kd * derivativo;

    //saturación de la salida
    if (output > pid->max_output) {

```

```

    output = pid->max_output;
}
if (output < pid->min_output){
    output = pid->min_output;
}

output2 = output ;
if(output2 < 0){
    output2 *= -1;
}

//voltageOutput = errorpid;

voltageOutput = (output2 * 4095 )/ pid-> max_output ;

    if(output < 0){
        output *= -1;

        if(MotorVariable == 1){
            //Left Front
            LeftFront_back();
        }
        else if(MotorVariable == 2){
            //Right Front
            RightFront_back();
        }
        else if(MotorVariable == 3){
            // Left Back
            LeftBack_back();
        }
        else if(MotorVariable == 4){
            // Right Back
            RightBack_back();
        }
    }

if(output > 0){
    if(MotorVariable == 1){
        //Left Front
        LeftFront_forward();
    }
    else if(MotorVariable == 2){
        //Right Front
        RightFront_forward();
    }
    else if(MotorVariable == 3){
        // Left Back
        LeftBack_forward();
    }
    else if(MotorVariable == 4){
        // Right Back
        RightBack_forward();
    }
}

```

/*

```

if(output < 0 && output > pid->min_output){
    voltageOutput = output * (-4095)/pid->Prev_output;

    if(MotorVariable == 1){
        //Left Front
        LeftFront_back();
    }
    else if(MotorVariable == 2){
        //Right Front
        RightFront_back();
    }
    else if(MotorVariable == 3){
        // Left Back
        LeftBack_back();
    }
    else if(MotorVariable == 4){
        // Right Back
        RightBack_back();
    }
}
else if(output > 0 && output < pid->max_output){
    voltageOutput = output*(4095.0/pid->max_output) + pid->Prev_output; //255 es
el max PWM, maxOutput es valor en PWM
}
*/

//actualizar para la próxima iteración
pid->errorAnterior = errorpid;
pid->medicionSensor_Anterior = medicionSensor;
pid->Prev_output = voltageOutput;
    return voltageOutput + pid->Prev_output;

}

void PIDAngle_Init(PIDpwm_Controller *pid, float Kp, float Ti, float min_output, float
max_output){
    pid->Kp = Kp;
    pid->Ki = 0;//Kp/Ti;

    //inicializar variables tipo float
    pid->setpoint = 0.0f;
    pid->integral = 0.0f;
    pid->errorAnterior = 0.0f;

    //limites del PID
    pid->min_output = min_output;
    pid->max_output = max_output;
}

float PIDAngle_Compute(PIDpwm_Controller *pid, float medicionAngulo){
    if( -1.2 < medicionAngulo && medicionAngulo < 1.2 ){
        medicionAngulo = pid->setpoint;
    }
    float error = pid->setpoint - medicionAngulo;
    pid->integral += error*0.05;

```

```

float output = pid->Kp * error + pid->Ki * pid->integral;

//actualizar para la próxima iteración
pid->medicionSensor_Anterior = medicionAngulo;

//saturación de la salida
if (output > pid->max_output) output = pid->max_output;
if (output < pid->min_output) output = pid->min_output;

//Conversion
float voltageOutput = output*(4095.0/pid->max_output); //4095 es el max PWM,
maxOutput es valor en PWM
pid->Prev_output = voltageOutput;
return voltageOutput + pid->Prev_output;
}

```

Code 2 Library to compute control values of the vehicle.

```

/*
 * Directions.c
 *
 * Created on: Nov 5, 2023
 * Author: Jorge
 */

#include "main.h"

void Forward(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 1); //izq atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0); //izq atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 1); //derecha atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, 0); //derecha atrasa
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 1); //derecha enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 0); //derecha enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 1); //izq enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 0); //ezq enfrente
}

void RightTurn(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 1);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, 1);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 0);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 1);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 0);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 1);
}

void Stop(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 0);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 0);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 0);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 0);
}

```

```

void Back(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 1); //izq atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0); //izq atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 0); //derecha atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, 1); //derecha atrasa
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 1); //derecha enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 0); //derecha enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 0); //izq enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 1); // ezq enfrente
}

void LeftTurn(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 1);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 1);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, 0);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 1);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 0);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 1);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 0);
}

void RightFront_back(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 0); //derecha enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 1); //derecha enfrente
}

void LeftFront_back(void) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 0); //izq enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 1); // ezq enfrente
}

void RightBack_back(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 0); //derecha atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, 1); //derecha atrasa
}

void LeftBack_back(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 0); //izq atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 1); //izq atras
}

void RightFront_forward(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, 1); //derecha enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, 0); //derecha enfrente
}

void LeftFront_forward(void) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, 1); //izq enfrente
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 0); // ezq enfrente
}

void RightBack_forward(void) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 1); //derecha atras
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, 0); //derecha atrasa
}

```

```
void LeftBack_forward(void) {  
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, 1); //izq atras  
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, 0); //izq atras  
}
```

Code 3 Library to set directions of the vehicle.